24

DEEP DIVE 24 · పది CONNECTIONS, వెయ్యి REQUESTS

Design a Connection Pool

> ప్రతి web app · ప్రతి microservice · HikariCP, pgbouncer, database/sql – "connections ని ఒక జాబితాలో పెట్టి తిరిగి వాడదాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది pool ఖాళీ అయినప్పుడు – **undefined**, ఆ తర్వాత "ఎదురుచూద్దాం" అనే సహజ పరిష్కారం ఆలస్యాన్ని **1,990 ms** కి తీసుకెళ్తుంది, ఇంకా పెరుగుతూనే. రెండోది – కేవలం **9 లీక్‌లు 52% requests ని** చంపుతాయి. మూడోది ఒక నిశ్శబ్ద రాత్రి తర్వాత **750 queries విఫలం**, సరిగ్గా రద్దీ మొదలైన క్షణంలో.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి output, ప్రతి సంఖ్య నిజంగా `node` lo run చేసినదే.**

DEEP DIVE 22 (LOGGING FRAMEWORK) చదివారా? – ఇది ఆ QUEUE కథ కాదు

DD 22 lo `LevelQueue` చూశాం – queue నిండితే తక్కువ ముఖ్యమైన వాటిని **వదిలేయడం**. ఆ ఆలోచన ఇక్కడ §5 lo మళ్ళీ వస్తుంది, కాబట్టి దాన్ని మళ్ళీ మొదటి నుంచి వివరించను.

Connection pool యొక్క నిజమైన problem పూర్తిగా వేరేది, మరియు అది ఒక్క వాక్యంలో ఉంది:

ఒక log line ని వదిలేయడం *ఉచితం*. ఒక connection ని వదిలేయడం *శాశ్వతం*.

Logging lo queue lo ఉన్నది ఒక *సందేశం* – పోతే పోయింది, system ఆరోగ్యంగానే ఉంటుంది. ఇక్కడ queue lo ఉన్నది ఒక **ఎదురుచూస్తున్న user**, మరియు వనరు **తిరిగి రావాలి**. తిరిగి రాకపోతే అది ఎప్పటికీ పోయినట్లే – మరియు ఏ programming language కూడా ఆ తిరిగి రావడాన్ని బలవంతం చేయదు. అదే §7.

మరియు §10 – DD 22 lo appender ఎప్పుడూ చావదు. ఇక్కడ connection **మనకు చెప్పకుండా** చచ్చిపోతుంది, మరియు అది చచ్చిందని మనకు తెలిసేది **user ఒక query పంపినప్పుడే**.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు pool అసలు ఎందుకు ఉంది?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: pool ఖాళీ అయితే

3. Step – ఒక జాబితా · `borrow` మరియు `release`
4. **మొదటి విరుపు** – `undefined`, ఆ తర్వాత అంతులేని ఎదురుచూపు
5. Step – గడువు · "లేదు" అని చెప్పగలగడం

Part 3 – రెండో విరుపు: తిరిగి రాని connections

6. Step – బయట ఎన్ని ఉన్నాయో లెక్కపెట్టడం
7. **రెండో విరుపు** – 9 లీక్‌లు, సగం system చచ్చిపోయింది
8. Step – `use()` · మరియు నా reaper చేసిన తప్పు

Part 4 – మూడో విరుపు: చచ్చిన connections

9. Step – pool ని రాత్రంతా ఉంచడం
10. **మూడో విరుపు** – 750 వైఫల్యాలు, సరిగ్గా రద్దీ మొదలైన క్షణంలో
11. Step – ఖాళీగా కూర్చున్నదాన్ని మాత్రమే పరీక్షించడం

Part 5 – పూర్తి system

12. Step – pool పరిమాణం ఎంత ఉండాలి? (పెద్దది చేస్తే **ఘోరం**)
13. మొత్తం code · 3,000 యాదృచ్ఛిక ప్రయోగాలు · **నాలుగు నియమాలు**

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటితో చెప్పాల్సిన English script
16. Follow-ups – circuit breaker, బహుళ databases, async/await
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. అడిగింది ఏమిటి – మరియు pool అసలు ఎందుకు ఉంది?

"Database connection pool design చెయ్యి" అంటే చాలామంది వెంటనే `Array` మరియు `pop()` గురించి మాట్లాడతారు. కానీ ముందు **ఈ వస్తువు అసలు ఎందుకు ఉంది** అన్నది స్పష్టంగా చెప్పాలి – ఎందుకంటే మిగతా ప్రతి నిర్ణయం దాని నుంచే వస్తుంది.

ఒక database connection తెరవడం ఖరీదైనది: TCP handshake, TLS, తర్వాత authentication. అది సుమారు **30 ms**. అసలు query **2 ms**. అంటే ప్రతి query కి కొత్త connection తెరిస్తే, పనిలో **94% సమయం తెరవడానికే** పోతుంది.

> **ఇక్కడ 30 MS మరియు 2 MS నేను ఊహించిన ఖర్చులు – అవి మీ NETWORK మీద ఆధారపడతాయి. కానీ వాటి పర్యవసానం కొలిచినదే:**

```
1,000 queries · ఒక్కో connection తెరవడానికి 30 ms · ఒక్కో query 2 ms

  విధానం                    | తెరిచిన connections | సగటు latency | మొత్తం సమయం
  --------------------------+---------------------+--------------+-------------
  ప్రతి query కీ కొత్తది    |                1000 |      32.0 ms |     3029 ms
  pool (10)                 |                  10 |       2.0 ms |     2999 ms
```

16× తక్కువ latency, 100× తక్కువ connections. ఇదే pool ఉనికికి కారణం.

కానీ రెండో కారణం ఇంకా ముఖ్యమైనది, మరియు అది interview lo చాలా తక్కువమంది చెప్తారు:

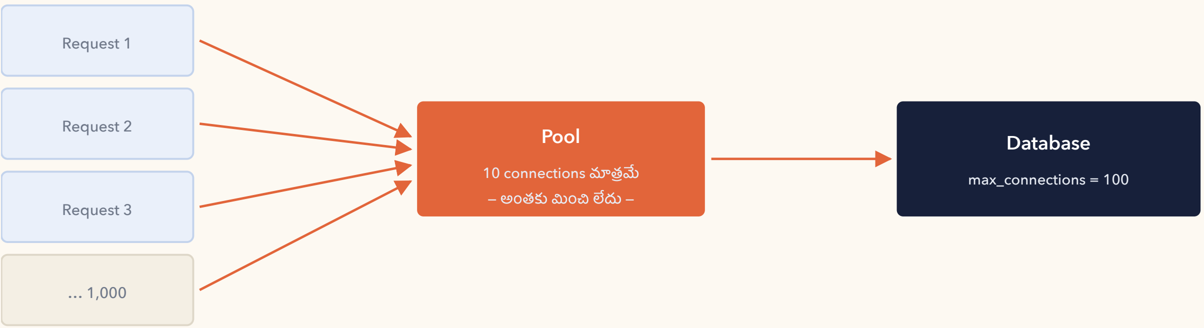
POOL యొక్క రెండో పని – మరియు అసలు ముఖ్యమైనది

Database కి **ఒక గరిష్ఠ పరిమితి** ఉంటుంది (Postgres lo `max_connections`, సాధారణంగా 100). అది దాటితే database *కొత్త connections ని తిరస్కరించడం* మొదలుపెడుతుంది – అంటే **అప్పటికే నడుస్తున్న** queries తో సహా అంతా చెడిపోతుంది.

కాబట్టి pool అనేది కేవలం **వేగం కోసం cache కాదు**. అది మీ app మరియు database మధ్య ఒక **అడ్డుకట్ట (limiter)**. అది database ని మీ స్వంత traffic నుంచి కాపాడుతుంది.

ఈ రెండో పనే §12 lo "pool ని పెద్దది చేస్తే ఘోరం" అనే ఫలితానికి కారణం.

Pool ఎక్కడ కూర్చుంటుంది – మరియు అది దేన్ని కాపాడుతుంది



వెయ్యి requests లోపలికి · **పది** బయటికి. ఆ సంకోచమే pool యొక్క అసలు విలువ.

కాబట్టి మనం design చేస్తున్నది: పరిమిత సంఖ్యలో ఖరీదైన, **తిరిగి వాడదగిన** వస్తువులను పంచే ఒక వ్యవస్థ – అవి **తప్పక తిరిగి రావాలి**, మరియు అవి **మనకు చెప్పకుండా చచ్చిపోగలవు**.

---

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Interview lo ఈ ప్రశ్నలు అడగడం వల్ల మీరు ఈ problem ని ఇంతకుముందు ఆలోచించారని తెలుస్తుంది. ప్రతిదానికీ **ఎందుకు అడుగుతున్నామో** కూడా ఇచ్చాను – interviewer అదే వినాలనుకుంటాడు.

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Pool ఖాళీ అయితే ఏమి జరగాలి – ఎదురుచూడాలా, తిరస్కరించాలా?	ఇదే §4/§5 మొత్తం. "ఎదురుచూడు" అనేది సమాధానం కాదు, అది **ఆలస్యాన్ని అపరిమితం** చేస్తుంది
ఎదురుచూసేవాళ్ళ క్రమం FIFO నా?	FIFO కాకపోతే కొందరు **ఎప్పటికీ** connection పొందరు (starvation)
Connection ని తిరిగి ఇవ్వడం మర్చిపోతే?	లీక్. §7 lo 9 లీక్‌లు 52% system ని చంపుతాయి
Database restart అయితే pool కి తెలుస్తుందా?	తెలియదు. §10
Pool పరిమాణం ఎవరు నిర్ణయిస్తారు – మనమా, DBA యా?	సమాధానం DBA. §12 lo ఎందుకో చూస్తాం
ఒకే database యా, read-replica కూడా ఉందా?	ఉంటే pool ఒకటి కాదు, **ఒక్కో లక్ష్యానికి ఒకటి**. §16

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Transaction మధ్యలో connection మారవచ్చా?	**కూడదు** – transaction అంతా ఒకే connection మీదే. ఇది `use()` design ని నిర్ణయిస్తుంది

ఒక ప్రశ్న అడగకూడదు

"Thread-safe గా ఉండాలా?" అని అడగవద్దు – **అవును**, ఎప్పుడూ. Pool అంటేనే పంచుకునే వస్తువు. అది అడిగితే మీరు ఈ problem ని ఇంకా అర్థం చేసుకోలేదని అర్థం.

బదులుగా చెప్పండి: *"JavaScript single-threaded కాబట్టి ఇక్కడ lock అవసరం లేదు, కానీ Java lo ఇదే code కి `free` జాబితా మరియు `waiters` రెండింటి మీదా ఒకే lock కావాలి – లేదా `ArrayBlockingQueue`."* అది మీకు రెండూ తెలుసని చూపిస్తుంది.

Part 2 – మొదటి విరుపు: pool ఖాళీ అయితే

3. Step – ఒక జాబితా · `borrow` మరియు `release`

మొదట అందరూ రాసే code. ఇది తప్పు కాదు – ఇది **అసంపూర్ణం**, మరియు ఎక్కడ అసంపూర్ణమో చూడటమే ఈ Part.

```
class NaivePool {
  constructor(size) {
    this.free = [];
    for (let i = 0; i < size; i++) this.free.push({ id: i + 1 });
  }
  borrow()  { return this.free.pop(); }
  release(c) { this.free.push(c); }
}
```

మూడు పంక్తులు. `borrow` ఒకటి తీస్తుంది, `release` వెనక్కి పెడుతుంది. నడిపి చూద్దాం:

```
const pool = new NaivePool(3);
console.log('  pool పరిమాణం : 3\n');

const a = pool.borrow(), b = pool.borrow(), c = pool.borrow();
console.log('  మూడూ తీసుకున్నాం :', a.id + ',', b.id + ',', c.id);

const d = pool.borrow();            // ← నాలుగోది
console.log('  నాలుగోది          :', d);

console.log('\n  → undefined. తర్వాతి పంక్తి ఏమవుతుంది?');
try { d.query('SELECT 1'); }
catch (e) { console.log('   ', e.constructor.name + ':', e.message); }
```

```
  pool పరిమాణం : 3

  మూడూ తీసుకున్నాం : 3, 2, 1
  నాలుగోది          : undefined

  → undefined. తర్వాతి పంక్తి ఏమవుతుంది?
    TypeError: Cannot read properties of undefined (reading 'query')
```

4. మొదటి విరుపు – `undefined`, ఆ తర్వాత అంతులేని ఎదురుచూపు

ఆ `TypeError` ని జాగ్రత్తగా చదవండి. అది **అబద్ధం చెప్తోంది**.

నిజమైన సమస్య "connection మీద `query` లేదు" కాదు. నిజమైన సమస్య **"మా database సామర్థ్యం అయిపోయింది"**. కానీ ఆ TypeError ని చూసిన developer రాత్రంతా `query` method ఎక్కడ పోయిందా అని వెతుకుతాడు.

> **పాఠం – సామర్థ్య లోపాన్ని `UNDEFINED` గా అనువదించకూడదు**
>
> `undefined` తిరిగి ఇవ్వడం వల్ల **ఏమి జరిగిందో** అనే సమాచారం పూర్తిగా పోతుంది. Stack trace వాడిన చోట చూపిస్తుంది, *అసలు కారణం* దగ్గర కాదు.
>
> ఇది ఈ doc lo పదేపదే వచ్చే విషయం: **సమస్య పుట్టిన చోట దాని పేరు చెప్పాలి.**

సరే, మరి ఏమి చెయ్యాలి? సహజమైన సమాధానం అందరికీ వెంటనే వస్తుంది: **"ఖాళీ లేకపోతే ఒకటి ఖాళీ అయ్యేవరకు ఎదురుచూడు."**

దాన్ని రాద్దాం:

```
class WaitingPool {
  constructor(sim, size) {
    this.sim = sim; this.free = []; this.waiters = [];
    for (let i = 0; i < size; i++) this.free.push({ id: i + 1 });
  }
  borrow(cb) {
    if (this.free.length) return cb(this.free.pop());
    this.waiters.push(cb);                    // ఎదురుచూసేవాళ్ళ వరుస
  }
  release(c) {
    const w = this.waiters.shift();           // FIFO — మొదట అడిగినవాడికి మొదట
    if (w) return w(c);                       // connection ని నేరుగా అప్పగించు
    this.free.push(c);
  }
}
```

ఇది మెరుగ్గా అనిపిస్తుంది. `undefined` లేదు, `TypeError` లేదు, ఎవరూ తిరస్కరించబడలేదు. **ఇదే చాలా interviews lo "సరైన" సమాధానంగా చెప్పబడుతుంది.**

కొలిచి చూద్దాం. Pool 10, ప్రతి 5 ms కి ఒక request:

```
pool 10 · ప్రతి 5 ms కి ఒక request · ఒక్కో request 40 ms పడుతుంది

  (10 connections × 5 ms = 50 ms సామర్థ్యం · demand 40 ms — సరిపోతుంది)

  arrivals |  served |  p50 wait |  p99 wait |  గరిష్ఠ wait
  ---------+---------+-----------+-----------+--------------
       500 |     500 |      0 ms |      0 ms |         0 ms
      2000 |    2000 |      0 ms |      0 ms |         0 ms
```

అద్భుతం. ఒక్క ms కూడా ఎదురుచూపు లేదు, 2,000 requests అయినా ఫరవాలేదు.

ఇప్పుడు **ఒక్కటే మార్పు**: requests కొంచెం నెమ్మదిగా అవుతాయి – 40 ms కి బదులు **60 ms**. అంటే demand సామర్థ్యాన్ని కేవలం 20% దాటింది:

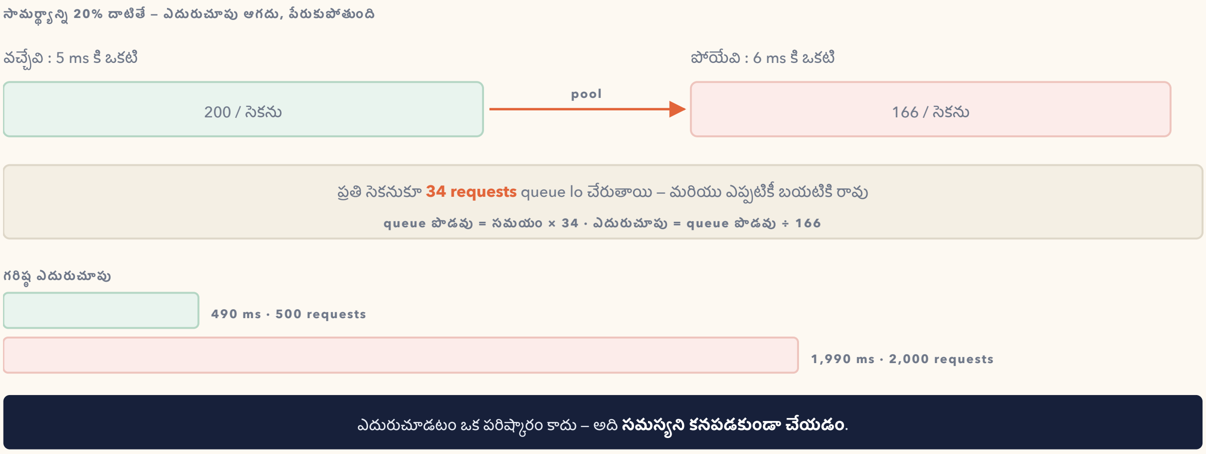
```
arrivals |  served |  p50 wait |  p99 wait |  గరిష్ఠ wait
---------+---------+-----------+-----------+--------------
     500 |     500 |    250 ms |    490 ms |       490 ms
    2000 |    2000 |   1000 ms |   1980 ms |      1990 ms
```

ఈ రెండు వరుసలని పక్కపక్కన చూడండి – ఇదే మొదటి విరుపు

500 requests → గరిష్ఠ ఎదురుచూపు **490 ms**.
2,000 requests → గరిష్ఠ ఎదురుచూపు **1,990 ms**.

4× ఎక్కువ requests → 4× ఎక్కువ ఎదురుచూపు. ఇది స్థిరపడదు. 20,000 requests వస్తే 20 సెకన్లు. 2,00,000 వస్తే 200 సెకన్లు.

మరియు గమనించండి – **served ఎప్పుడూ 100%**. ప్రతి request "విజయవంతమైంది". మీ error rate dashboard **పచ్చగా** ఉంటుంది. కానీ మీ users 2 సెకన్లు ఎదురుచూస్తున్నారు, మరియు అది పెరుగుతూనే ఉంది.



DD 22 తో సంబంధం

DD 22 §9 lo **అపరిమిత queue** memory ని తినేస్తుందని చూశాం, మరియు దానికి పరిమితి పెట్టాం. ఇక్కడ అదే వ్యాధి, కానీ లక్షణం వేరు – **memory కాదు, సమయం** పెరుగుతోంది.

ఎందుకంటే ఇక్కడ queue lo ఉన్నది ఒక సందేశం కాదు, **ఒక వ్యక్తి**. ఆ వ్యక్తికి browser tab తెరిచి ఉంది, మరియు అతని HTTP request కి కూడా ఒక గడువు ఉంది.

---

5. Step – గడువు · "లేదు" అని చెప్పగలగడం

పరిష్కారం ఒక్క మాటలో: **ఎదురుచూపుకి ఒక గడువు పెట్టాలి, మరియు గడువు దాటితే స్పష్టంగా తిరస్కరించాలి.**

```
class Pool {
  constructor(sim, size, { timeoutMs = Infinity } = {}) {
    this.sim = sim; this.free = []; this.waiters = [];
    this.timeoutMs = timeoutMs;
    for (let i = 0; i < size; i++) this.free.push({ id: i + 1 });
  }
  borrow(cb) {
    if (this.free.length) return cb(null, this.free.pop());
    const w = { cb, dead: false };
    this.waiters.push(w);
    if (this.timeoutMs === Infinity) return;
    this.sim.after(this.timeoutMs, () => {
      if (w.dead) return;                          // ఈలోపు connection దొరికింది
      w.dead = true;
      this.waiters.splice(this.waiters.indexOf(w), 1);
      cb({ reason: 'POOL_TIMEOUT' }, null);        // స్పష్టమైన తిరస్కరణ
    });
  }
  release(c) {
    while (this.waiters.length) {
      const w = this.waiters.shift();
      if (w.dead) continue;                        // గడువు దాటినవాడిని దాటేయ్
      w.dead = true;
      return w.cb(null, c);
    }
    this.free.push(c);
  }
}
```

రెండు చిన్న విషయాలు ముఖ్యమైనవి:

ఒకటి – `dead` అనే గుర్తు. ఒక waiter కి గడువు దాటింది అని చెప్పాక, ఇంకెప్పుడూ అతనికి connection ఇవ్వకూడదు. `dead` లేకపోతే `release` అతనికి connection అప్పగిస్తుంది – కానీ అతని callback అప్పటికే "విఫలం" అని చెప్పేసింది. ఆ connection **అక్కడే పోతుంది** (§7 lo చూసే లీక్).

రెండు – `release` lo `while`, `if` కాదు. `waiters` జాబితాలో చచ్చిన waiters ఉండొచ్చు. `if` వాడితే చచ్చినవాడిని తీసి connection ని పారేస్తాం. `while` బతికి ఉన్నవాడు దొరికేవరకు వెతుకుతుంది.

ఇప్పుడు వేర్వేరు గడువులతో కొలిచి చూద్దాం – సామర్థ్యం కంటే ఎక్కువ demand ఉన్నప్పుడు:

```
pool 10 · ప్రతి 5 ms కి request · ఒక్కో request 60 ms (సామర్థ్యం కంటే ఎక్కువ)

  గడువు      |  served |  తిరస్కృతం |  p50 wait |  p99 wait |  గరిష్ఠ wait
  -----------+---------+------------+-----------+-----------+--------------
  లేదు       |    2000 |          0 |   1000 ms |   1980 ms |      1990 ms
  500 ms     |    1750 |        250 |    500 ms |    500 ms |       500 ms
  200 ms     |    1700 |        300 |    200 ms |    200 ms |       200 ms
  100 ms     |    1684 |        316 |    100 ms |    100 ms |       100 ms
```

ఈ పట్టికలో ముఖ్యమైనది "తిరస్కృతం" నిలువు వరుస కాదు – "గరిష్ఠ WAIT" వరుస

గడువు లేకపోతే గరిష్ఠ ఎదురుచూపు **1,990 ms**, మరియు అది arrivals తో పెరుగుతుంది.

గడువు పెడితే గరిష్ఠ ఎదురుచూపు **సరిగ్గా గడువే** – 500, 200, 100. అది ఎన్ని requests వచ్చినా మారదు. **మీరు latency మీద ఒక పైకప్పు పెట్టారు.**

మరియు 100 ms కీ 500 ms కీ మధ్య served లో తేడా కేవలం **66 requests** (1,684 vs 1,750) – 4% మాత్రమే. కానీ latency **5× తక్కువ**. అంటే **పొడవైన గడువు దాదాపు ఏమీ కొనదు.**

గడువు ఎంత పెట్టాలి? – ఒక నియమం

Pool గడువు **మీ HTTP request గడువు కంటే తక్కువ** ఉండాలి. లేకపోతే ఇలా జరుగుతుంది: user యొక్క browser 2 సెకన్ల దగ్గర request ని వదిలేస్తుంది, కానీ మీ pool ఇంకా 3వ సెకనులో connection వెతుకుతూనే ఉంటుంది – **ఎవరూ లేని పని కోసం ఒక connection ఖర్చు.**

ఇది నిజమైన systems lo చాలా సాధారణమైన తప్పు, మరియు interview lo దీన్ని చెప్తే గుర్తుండిపోతారు.

మొదటి విరుపు పూర్తయింది. ఇప్పుడు pool ఖాళీ అయితే ఏమి జరుగుతుందో మనకు తెలుసు, మరియు దాన్ని మనమే నియంత్రిస్తున్నాం.

కానీ ఈ code lo ఒక **మౌనమైన** ఊహ ఉంది, మరియు అదే తర్వాతి విరుపు.

Part 3 – రెండో విరుపు: తిరిగి రాని connections

6. Step – బయట ఎన్ని ఉన్నాయో లెక్కపెట్టడం

మన pool మొత్తం ఒక్క ఊహ మీద నిలబడి ఉంది:

> **`borrow()` చేసిన ప్రతి ఒక్కరూ చివరికి `release()` చేస్తారు.**

ఈ ఊహని ఎవరూ అమలు చేయడం లేదు. `free.pop()` చేశాక ఆ connection ఎక్కడికి పోయిందో pool కి **అస్సలు తెలియదు**. అది కేవలం "ఇక్కడ లేదు" అని మాత్రమే తెలుసు.

నిజమైన code ఇలా ఉంటుంది:

```
// ఇది నిజమైన applications lo ప్రతిచోటా కనిపించే ఆకారం
function handleRequest(req, res) {
  pool.borrow((err, conn) => {
    if (err) return res.status(503).send('busy');
    const user = conn.query('SELECT * FROM users WHERE id = ?', req.id);
    if (!user) return res.status(404).send('not found');   // ← ఇక్కడ చూడండి
    res.send(user);
    pool.release(conn);
  });
}
```

ఆ `return res.status(404)` పంక్తిని చూడండి. User దొరకకపోతే function అక్కడే తిరిగి వెళ్ళిపోతుంది – `pool.release(conn)` **ఎప్పటికీ నడవదు.**

ఇది ఎవరో నిర్లక్ష్యంగా రాసిన code కాదు. ఇది **సహజమైన, శుభ్రంగా కనిపించే** code. Early return ఒక మంచి అలవాటు. కానీ ఇక్కడ అది ఒక connection ని శాశ్వతంగా మింగేస్తుంది.

అదే `throw` తోనూ, `await` విఫలమైనా, timeout తోనూ జరుగుతుంది.

లీక్ అంటే ఏమిటి – మరియు అది ఎందుకు ప్రత్యేకమైనది

Memory leak lo మీరు memory కోల్పోతారు, కానీ garbage collector చాలావరకు కాపాడుతుంది.

Connection leak **తిరిగి రాదు**. ఆ object ఇంకా JavaScript lo బతికే ఉంది (మీ closure దాన్ని పట్టుకుని ఉంది), కాబట్టి GC దాన్ని ముట్టుకోదు. Database వైపు ఆ connection ఇంకా తెరిచే ఉంది. కేవలం **pool కి అది ఎప్పటికీ తిరిగి రాదు.**

Pool పరిమాణం 10 అయితే, **10 లీక్‌లతో మీ app శాశ్వతంగా చచ్చిపోతుంది** – restart చేసేవరకు.

దీన్ని కనీసం **చూడగలగాలి** – అందుకే బయట ఎన్ని ఉన్నాయో లెక్కపెడదాం:

```
stats() {
  return {
    free:  this.free.length,        // pool lo ఉన్నవి
    inUse: this.out.size,           // బయట ఉన్నవి
    live:  this.live.length,        // మొత్తం తెరిచినవి
  };
}
```

`free + inUse === live` – ఇది ఎప్పుడూ నిజం కావాలి. ఇదే మన మొదటి **నియమం (invariant)**, మరియు §13 lo దీన్ని 3,000 యాదృచ్ఛిక ప్రయోగాలతో పరీక్షిస్తాం.

---

7. రెండో విరుపు – 9 లీక్‌లు, సగం system చచ్చిపోయింది

ఇప్పుడు కొలుద్దాం. Pool 10, 4,000 requests, ఒక్కో request 40 ms. ప్రతి కొన్నీ request ఒక exception విసురుతుంది మరియు `release` జరగదు.

నా ఊహ ఏమిటంటే: లీక్ రేటు తక్కువగా ఉంటే ప్రభావం తక్కువగా ఉంటుంది. **అది పూర్తిగా తప్పు.**

```
pool 10 · 4,000 requests · ఒక్కో request 40 ms

  లీక్ రేటు      | లీక్‌లు | పూర్తయినవి | తిరస్కృతం | విజయం % | pool చచ్చిన క్షణం
  --------------+-------+----------+-----------+---------+------------------
  లేదు            |     0 |    4000 |         0 |    100% |          –
  ప్రతి 1000వ    |     3 |    3910 |        90 |     98% |   16635 ms
  ప్రతి 500వ     |     6 |    3252 |       748 |     81% |    9135 ms
  ప్రతి 200వ     |     9 |    1912 |      2088 |     48% |    4395 ms
```

చివరి వరుసని మళ్ళీ చదవండి

9 లీక్‌లు. 4,000 requests lo కేవలం తొమ్మిది.

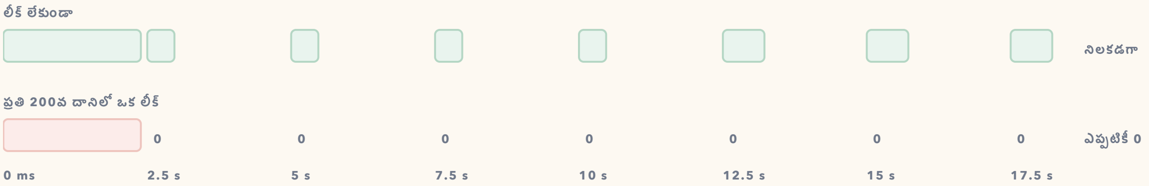
ఫలితం: **2,088 requests తిరస్కరించబడ్డాయి** (52%), మరియు pool **4.4 సెకన్లకే** పూర్తిగా చచ్చిపోయింది.

ఇది 0.22% లోపం రేటు. అది **52% వైఫల్య రేటు** గా మారింది – **236 రెట్లు పెద్దది.**

ఎందుకు ఇంత తీవ్రం? ఎందుకంటే లీక్ ఒక **శాశ్వత నష్టం**, తాత్కాలికమైనది కాదు.

ఒక నెమ్మదైన query pool ని కొద్దిసేపు ఇబ్బంది పెడుతుంది, తర్వాత connection తిరిగి వస్తుంది. ఒక లీక్ ఆ connection ని **ఎప్పటికీ** తీసేస్తుంది. Pool యొక్క సామర్థ్యం 10 నుంచి 9 కి, 8 కి, 7 కి **ఒక దిశలో మాత్రమే** దిగుతుంది.

pool lo ఖాళీగా ఉన్న connections – కాలక్రమంలో



రెండు వరుసల్లోనూ మొదటి క్షణంలో 10 ఖాళీ connections ఉన్నాయి.

కింది వరుసలో అవి **తిరిగి రాలేదు** – 2.5 సెకన్లకే pool ఖాళీ, మరియు అది శాశ్వతం.

నెమ్మదైన query = **తాత్కాలిక** ఒత్తిడి. లీక్ = **శాశ్వత** సామర్థ్య నష్టం.

అందుకే 0.22% లోపం 52% వైఫల్యం అవుతుంది.

INTERVIEW LO ఇది ఎందుకు బలమైన క్షణం

చాలామంది "connection leak చెడ్డది" అని చెప్తారు. మీరు **"pool 10 తో, 4,000 requests lo 9 లీక్‌లు 52% traffic ని చంపుతాయి, మరియు pool 4.4 సెకన్లకే చచ్చిపోతుంది"** అని చెప్పగలిగితే – అది పూర్తిగా వేరే స్థాయి సమాధానం.

ఎందుకంటే అది **లీక్ ఒక సాధారణ bug కాదు, అది ఒక రకమైన మరణం** అని చూపిస్తుంది.

---

8. Step – `use()` · మరియు నా reaper చేసిన తప్పు

రెండు పరిష్కారాలు కావాలి, మరియు **రెండూ** కావాలి.

పరిష్కారం ఒకటి – `release` ని మర్చిపోవడం సాధ్యం కాకుండా చేయడం

అసలు సమస్య `release` ని *పిలవడం మర్చిపోవడం* కాదు. సమస్య ఏమిటంటే **API అది మర్చిపోవడాన్ని అనుమతిస్తోంది**.

కాబట్టి `borrow` / `release` ని బహిరంగ API గా ఇవ్వకూడదు. బదులుగా `use()` ఇవ్వాలి – అది `release` ని **మీ తరఫున** చేస్తుంది:

```
use(work, done = () => {}) {
  this.borrow((err, c) => {
    if (err) return done(err, null);
    let released = false;
    const finish = (e, v) => {
      if (released) return;                    // రెండోసారి పిలిస్తే పట్టించుకోవద్దు
      released = true;
      this.release(c, { broken: !!e });
      done(e, v);
    };
    try { work(c, finish); }
    catch (e) { finish(e, null); }             // విసిరినా connection వెనక్కి
  });
}
```

మూడు విషయాలు ఇక్కడ జరుగుతున్నాయి:

`try/catch` – `work` ఒక exception విసిరినా connection వెనక్కి వెళ్తుంది. §6 lo చూసిన early-return bug ఇక్కడ సాధ్యం కాదు, ఎందుకంటే `release` ని పిలిచేది మీరు కాదు.

`released` **గుర్తు** – ఒకే connection ని రెండుసార్లు తిరిగి ఇవ్వడం **లీక్ కంటే ప్రమాదకరం**. అప్పుడు ఒకే connection ఇద్దరికి ఇవ్వబడుతుంది, మరియు ఒకరి query మధ్యలో ఇంకొకరి query వచ్చి కూర్చుంటుంది.

`broken: !!e` – పని విఫలమైతే ఆ connection మీద నమ్మకం పోయింది. అది transaction మధ్యలో ఆగిపోయి ఉండొచ్చు. దాన్ని తీసేసి కొత్తది తెరవడం సురక్షితం.

> **ఇదే వాస్తవ పాఠం – మరియు ఇది POOL కి మాత్రమే పరిమితం కాదు**
>
> **సరిగ్గా వాడటం మీద ఆధారపడే API ని ఇవ్వకండి. సరిగ్గా వాడటం తప్ప వేరే దారి లేని API ఇవ్వండి.**
>
> Python lo `with`, Java lo try-with-resources, Go lo `defer`, C++ lo RAII, Rust lo `Drop` – ప్రతి భాషా ఈ ఒక్క సమస్యకే ఒక ప్రత్యేక feature ఇచ్చింది. అది ఎంత ముఖ్యమో అది చెప్తుంది.

పరిష్కారం రెండు – అయినా ఎవరో లీక్ చేస్తారు

`use()` ఉన్నా, ఎవరో ఒకరు `borrow()` ని నేరుగా వాడతారు. లేదా `finish` ని పిలవని ఒక callback రాస్తారు. కాబట్టి **చివరి రక్షణ** కావాలి: చాలాసేపు బయట ఉన్న connection ని **బలవంతంగా వెనక్కి లాగడం**.

నేను మొదట ఇలా రాశాను:

```
#track(c) {
  this.out.set(c, this.sim.now);
  this.sim.after(this.reapAfterMs, () => {
    if (this.out.get(c) !== undefined) {      // ఇంకా బయటే ఉందా?
      this.reaped++;
      this.release(c);                        // బలవంతంగా వెనక్కి
    }
  });
}
```

కొలిస్తే ఫలితం **అద్భుతంగా** కనిపించింది – pool ఇక చావడం లేదు, 4,000 requests అన్నీ పూర్తయ్యాయి:

```
పరిష్కారం: 2 సెకన్ల కంటే ఎక్కువసేపు బయట ఉన్న connection ని బలవంతంగా వెనక్కి తీసుకోవడం

  లీక్ రేటు      | లీక్‌లు | పూర్తయినవి | తిరస్కృతం | విజయం % | pool చచ్చిన క్షణం
  --------------+-------+---------+-----------+---------+-----------------
  ప్రతి 500వ     |     8 |    4000 |         0 |    100% |         –
                         (బలవంతంగా వెనక్కి తీసుకున్నవి: 77)
  ప్రతి 200వ     |    20 |    4000 |         0 |    100% |         –
                         (బలవంతంగా వెనక్కి తీసుకున్నవి: 391)
```

100% విజయం. కానీ **ఆ చివరి సంఖ్యని చూడండి.**

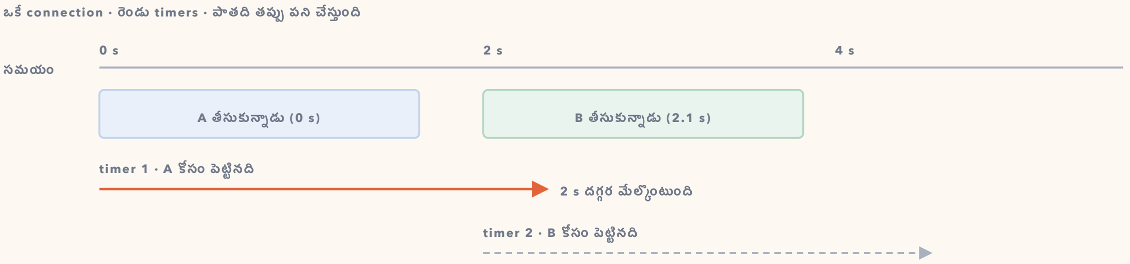
20 లీక్‌లు · 391 సార్లు "వెనక్కి లాగాం"

లీక్ అయినవి 20 మాత్రమే. మరి 391 ఎక్కడి నుంచి వచ్చాయి?

సమాధానం: **నా reaper లీక్ అవ్వని connections ని కూడా లాగేస్తోంది** – ఇప్పుడే ఇచ్చిన, చక్కగా పని చేస్తున్న వాటిని.

అంటే ఒక query నడుస్తుండగానే దాని connection ని లాగేసి ఇంకొకరికి ఇచ్చేస్తున్నాను. **ఇది లీక్ కంటే చాలా ఘోరమైన bug** – మరియు ఇది 100% విజయ రేటు వెనక దాక్కుంది.

కారణం ఇది. `release()` ఒక connection ని నేరుగా ఒక waiter కి అప్పగించినప్పుడు, నేను `#track(c)` మళ్ళీ పిలుస్తాను – అది **కొత్త timer** పెడుతుంది. కానీ **పాత timer ఇంకా ఆగిపోలేదు.** అది 2 సెకన్ల తర్వాత మేల్కొని "ఈ connection ఇంకా బయట ఉంది" అని చూస్తుంది – నిజమే, కానీ అది **వేరే ఎవరో** ఇప్పుడే తీసుకున్నది.



2.1 s దగ్గర timer 1 మేల్కొంటుంది. "ఈ connection బయటే ఉందా?" – **అవును**.

కానీ అది A ది కాదు – అది **B ది**, ఇప్పుడే ఇచ్చినది.

timer 1 దాన్ని లాగేస్తుంది. B ఇప్పుడు ఇంకొకరు వాడుతున్న connection మీద query చేస్తున్నాడు.

పరిష్కారం: "ఇది ఇంకా బయట ఉందా?" అని అడగడం సరిపోదు. **"ఇది ఇంకా ఆ checkout లోనే ఉందా?"** అని అడగాలి. ప్రతి checkout కి ఒక ప్రత్యేక గుర్తు ఇద్దాం:

```
#track(c) {
  const token = ++this.epoch;              // ఈ checkout కి ప్రత్యేక గుర్తు
  this.out.set(c, token);
  this.sim.after(this.reapAfterMs, () => {
    if (this.out.get(c) !== token) return; // వేరే checkout – ఈ timer పాతది
    this.out.delete(c);
    this.reaped++;
    this.#giveBack(c);
  });
}
```

ఒక్క పంక్తి మార్పు – `!== undefined` నుంచి `!== token`. ఇప్పుడు కొలిస్తే:

```
సరిదిద్దిన reaper (ప్రతి checkout కి ప్రత్యేక token) · pool 10 · 4,000 requests

  లీక్ రేటు      | లీక్‌లు | వెనక్కి లాగినవి | పూర్తయినవి | తిరస్కృతం | pool చచ్చిన క్షణం
  --------------+-------+------------+---------+-----------+------------------
  ప్రతి 500వ     |     8 |          8 |    4000 |         0 |          –
  ప్రతి 200వ     |    20 |         20 |    4000 |         0 |          –
```

8 లీక్‌లు → 8 లాగినవి. 20 → 20. సరిగ్గా సరిపోయాయి, ఒక్కటి ఎక్కువా కాదు.

ఈ తప్పు నుంచి అసలు పాఠం

నా మొదటి reaper **సరైన ఫలితాలు చూపించింది** – 100% విజయం, pool చావడం లేదు. నేను అక్కడే ఆగిపోయి ఉంటే, ఈ doc lo ఒక తీవ్రమైన bug ఉండేది.

అది బయటపడింది ఎందుకంటే నేను **reaper ఎన్నిసార్లు పని చేసిందో లెక్కపెట్టాను**, మరియు ఆ సంఖ్య **లీక్‌ల సంఖ్యకి సమానంగా ఉండాలని** నాకు తెలుసు.

ఫలితం సరిగ్గా ఉందా అని చూడటం సరిపోదు. అది సరైన కారణం వల్లే సరిగ్గా ఉందా అని చూడాలి. అందుకే ప్రతి మరమ్మత్తుకీ ఒక counter పెట్టండి, మరియు ఆ counter ఎంత ఉండాలో ముందే ఊహించండి.

లాగిన CONNECTION ని తిరిగి వాడకూడదు

గమనించండి – సరిదిద్దిన code lo `this.#giveBack(c)` ఉంది, కానీ **పూర్తి system lo (§13) దాన్ని మూసేసి కొత్తది తెరుస్తాం**.

ఎందుకంటే లీక్ అయిన connection మీద ఏమి జరిగిందో మనకు తెలియదు. అది ఒక తెరిచిన transaction మధ్యలో ఉండొచ్చు, లేదా చదవని ఫలితాలు దాని మీద పేరుకుని ఉండొచ్చు. దాన్ని ఇంకొకరికి ఇస్తే ఆ చెత్త అంతా వాళ్ళకి వెళ్తుంది.

Part 4 – మూడో విరుపు: చచ్చిన connections

9. Step – pool ని రాత్రంతా ఉంచడం

ఇప్పటివరకు మన pool ఒక ఊహ మీద నడుస్తోంది: `free` **జాబితాలో ఉన్న connection పని చేస్తుంది.**

ఈ ఊహ పగటిపూట నిజమే. రాత్రిపూట కాదు.

ఆలోచించండి – మీ app కి రాత్రి 2 గంటలకు traffic తక్కువ. 10 connections lo 9 **గంటల తరబడి ఖాళీగా కూర్చుంటాయి**. ఆ సమయంలో:

- Database ఒక maintenance restart చేసుకుంటుంది
- మీ cloud provider యొక్క NAT gateway నిశ్శబ్ద connections ని మూసేస్తుంది (AWS lo 350 సెకన్లు)
- ఒక firewall "ఇది ఎవరూ వాడటం లేదు" అని నిర్ణయించి మూసేస్తుంది
- Database యొక్క `idle_session_timeout` పని చేస్తుంది

ఈ అన్నిటిలోనూ **ఒకే విషయం** ఉమ్మడిగా ఉంది, మరియు అదే సమస్య:

> **TCP యొక్క ఒక క్రూరమైన నిజం**
>
> అవతలివైపు connection మూసేసినప్పుడు, **మీకు ఏమీ తెలియదు.**
>
> మీ వైపు socket object ఇంకా అక్కడే ఉంది. అది "connected" అనే చూపిస్తుంది. మీరు దాన్ని `free` జాబితాలో పెట్టారు, అది బాగానే కనిపిస్తోంది.
>
> అది చచ్చిందని మీకు తెలిసేది **మీరు దాని మీద ఒక query పంపినప్పుడే** – అంటే **ఒక నిజమైన user ఎదురుచూస్తున్నప్పుడు.**

ఇది కొలవడానికి ఒక వాస్తవిక కథ కావాలి, ఒక కృత్రిమమైనది కాదు. నా సిమ్యులేషన్ ఇలా ఉంది:

- **0-5 సెకన్లు:** పగటి రద్దీ – ప్రతి 5 ms కి ఒక request
- **5-65 సెకన్లు:** రాత్రి నిశ్శబ్దం – ప్రతి 6 సెకన్లకు ఒక request
- **firewall:** ప్రతి సెకనుకూ చూస్తుంది; **30 సెకన్ల కంటే ఎక్కువ ఖాళీగా ఉన్న** connection ని మూసేస్తుంది
- **65 సెకన్ల దగ్గర:** ఉదయం రద్దీ మళ్ళీ మొదలు

మొదటి ప్రయత్నంలో నా సిమ్యులేషన్ తప్పుగా ఉంది

నేను మొదట "ఒక క్షణంలో **అన్ని** ఖాళీ connections చచ్చిపోతాయి" అని రాశాను. అది 1,005 వైఫల్యాలు చూపించింది – పెద్ద సంఖ్య, కానీ **అన్యాయమైనది**: అది కేవలం 6 సెకన్లు ఖాళీగా ఉన్న connection ని కూడా చంపేస్తోంది.

నిజమైన firewall అలా చేయదు – అది **వయసు** చూసి చంపుతుంది. దాన్ని సరిచేశాక సంఖ్య 750 కి తగ్గింది, కానీ ఇప్పుడు అది **నిజమైన సంఖ్య**.

పెద్ద సంఖ్య కోసం సిమ్యులేషన్‌ని అన్యాయంగా ఉంచడం చాలా సులభం. అది doc ని బలహీనం చేస్తుంది, బలంగా కాదు.

---

10. మూడో విరుపు – 750 వైఫల్యాలు, సరిగ్గా రద్దీ మొదలైన క్షణంలో

```
firewall: 30 s కంటే ఎక్కువ ఖాళీగా ఉన్న connection మూత · 60 s నిశ్శబ్దం · pool 10

  వ్యూహం                      | విఫలం | మొదటి వైఫల్యం | pingలు | ping ఖర్చు
  ----------------------------+--------+----------------+---------+-----------
  పరీక్షించకపోవడం           |   750 |      65005 ms |       0 |       0 ms
```

ఆ `65005 ms` అనే సంఖ్యే ఈ విరుపు యొక్క అసలు కథ.

Firewall connections ని చంపింది **35 సెకన్ల దగ్గర**. మొదటి వైఫల్యం వచ్చింది **65 సెకన్ల దగ్గర** – ఉదయం రద్దీ మొదలైన క్షణం.

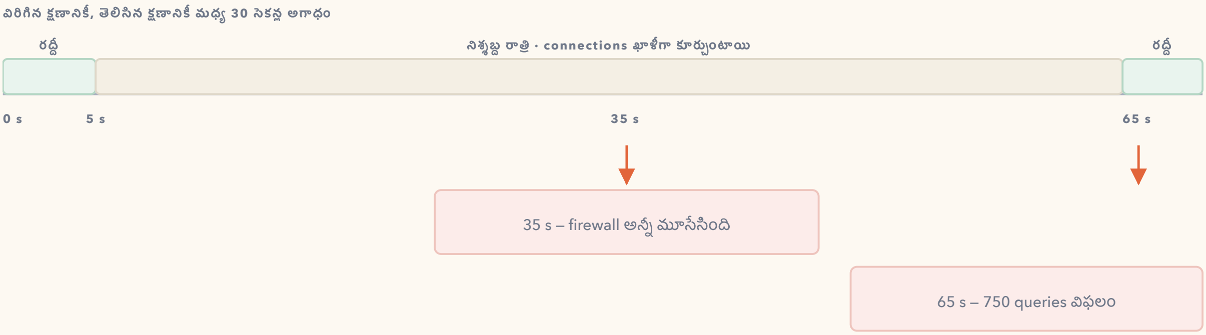
అంటే మీ system **30 సెకన్ల పాటు పూర్తిగా విరిగిపోయి ఉంది**, మరియు ఒక్క alert కూడా రాలేదు. Error rate సున్నా. ఎందుకంటే అడిగేవాళ్ళే లేరు.

ఇదే ఈ రకమైన BUG యొక్క సంతకం

మీ pool విరిగే **సమయానికీ**, మీకు తెలిసే **సమయానికీ** మధ్య ఒక అగాధం ఉంది. మరియు ఆ అగాధం చివర సరిగ్గా **traffic తిరిగి వచ్చే క్షణం** ఉంటుంది.

కాబట్టి మీ system ఎప్పుడూ ఇలాగే విరుగుతుంది: **సోమవారం ఉదయం 9 గంటలకి, ఆదివారం రాత్రి పడిన దెబ్బ వల్ల.**

ఇది కల్పన కాదు – ఇదే "Monday morning outage" అనే పేరుకి అసలు కారణం.



ఈ 30 సెకన్లలో error rate **సున్నా**. Dashboard పచ్చగా. Alert లేదు.
ఎందుకంటే system విరిగినట్టు తెలియాలంటే ఎవరో ఒకరు అడగాలి – మరియు ఎవరూ అడగలేదు.

సహజమైన పరిష్కారం, మరియు దాని ఖరీదు

"అయితే ఇవ్వబోయే ముందు ప్రతిసారీ ఒక చిన్న `SELECT 1` పంపుదాం" – ఇది సరైన ఆలోచన. కొలిచి చూద్దాం:

వ్యూహం	విఫలం	మొదటి వైఫల్యం	pingలు	ping ఖర్చు
పరీక్షించకపోవడం	750	65005 ms	0	0 ms
ప్రతిసారీ ping	0	–	2010	2010 ms

వైఫల్యాలు **సున్నా**. కానీ **2,010 pings** – ప్రతి ఒక్క borrow కీ ఒకటి.

అది ప్రతి query కి ఒక అదనపు network round-trip. మీ database 1 ms దూరంలో ఉంటే, **ప్రతి query 1 ms నెమ్మదైంది**. 2 ms query మీద అది **50% అదనపు ఖర్చు** – మనం §1 lo pool తో సంపాదించినదాన్ని తిరిగి ఇచ్చేసినట్టు.

మరియు ఇది ఒక వృథా ఖర్చు, ఎందుకంటే **ఇప్పుడే వాడిన connection చచ్చే అవకాశమే లేదు.**

---

11. Step – ఖాళీగా కూర్చున్నదాన్ని మాత్రమే పరీక్షించడం

ఇక్కడే అసలు అంతర్దృష్టి ఉంది, మరియు ఇది ఒక్క వాక్యంలో ఉంది:

> **Connections వాడటం వల్ల చావవు. ఖాళీగా కూర్చోవడం వల్ల చస్తాయి.**

Firewall, NAT, `idle_session_timeout` – అన్నీ **ఖాళీ సమయాన్ని** చూసి చంపుతాయి. కాబట్టి **ఖాళీ సమయం ఎక్కువ ఉన్నప్పుడు మాత్రమే** పరీక్షిద్దాం:

```
#check(c, cb) {
  const idleFor = this.sim.now - c.idleSince;
  if (idleFor <= this.idleValidateMs) return this.#hand(c, cb);   // ఖర్చు 0
  this.stats.validated++;
  if (c.alive) return this.#hand(c, cb);
  this.stats.replacedStale++;
  this.#retire(c);
```

```
  this.#hand(this.#make(), cb);                               // కొత్తది తెరువు
}
```

`release` lo ఒక్క పంక్తి జోడించాలి – connection ఎప్పుడు ఖాళీ అయిందో గుర్తుపెట్టుకోవడం:

```
release(c) {
  c.idleSince = this.sim.now;     // ఇప్పటి నుంచి ఇది ఖాళీగా కూర్చుంటోంది
  this.#giveBack(c);
}
```

ఇప్పుడు వేర్వేరు పరిమితులతో కొలుద్దాం:

```
firewall: 30 s కంటే ఎక్కువ ఖాళీగా ఉన్న connection మూత · 60 s నిశ్శబ్దం · pool 10

  వ్యూహం                    | విఫలం | మొదటి వైఫల్యం | pingలు | ping ఖర్చు
  --------------------------+-------+---------------+--------+-----------
  పరీక్షించకపోవడం          |   750 |      65005 ms |      0 |       0 ms
  ప్రతిసారీ ping            |     0 |             – |   2010 |    2010 ms
  25 s ఖాళీ అయితేనే ping    |     0 |             – |      3 |       3 ms
  10 s ఖాళీ అయితేనే ping    |     0 |             – |      3 |       3 ms
  5 s ఖాళీ అయితేనే ping     |     0 |             – |     13 |      13 ms
```

మూడో వరుస – ఈ DOC LO అత్యంత తృప్తికరమైన ఫలితం

విఫలాలు 0. Pings 3.

"ప్రతిసారీ ping" కూడా 0 వైఫల్యాలు ఇచ్చింది, కానీ దానికి **2,010 pings** అయ్యాయి. అదే రక్షణ, **670× తక్కువ ఖర్చుతో**.

ఎందుకంటే 2,010 borrows lo కేవలం **3** మాత్రమే నిజంగా ప్రమాదంలో ఉన్నాయి – రాత్రంతా ఖాళీగా కూర్చున్నవి. మిగతా 2,007 ఇప్పుడే వాడినవి.

పరిమితి ఎంత పెట్టాలి? పట్టికలో 25 s మరియు 10 s ఒకేలా ఉన్నాయి (3 pings), 5 s కి 13 – ఎక్కువ pings, ఒకే ఫలితం. కాబట్టి నియమం:

పరిమితి ఎంచుకోవడానికి నియమం

మీ పరిమితి **మీ మార్గంలో ఉన్న అన్ని timeouts కంటే తక్కువ** ఉండాలి – NAT యొక్క idle timeout, firewall యొక్క, database యొక్క `idle_session_timeout`. వాటిలో **అతి తక్కువది** తీసుకుని, దానిలో సగం పెట్టండి.

AWS NAT gateway 350 సెకన్లు → పరిమితి 150 సెకన్లు.

మరియు ఇంకొకటి – **ఇంకా తక్కువ పెట్టడం వల్ల ఏమీ లాభం లేదు**. పట్టికలో 5 s, 10 s రెండూ 0 వైఫల్యాలే. కేవలం pings పెరిగాయి.

ఇంకొక రక్షణ – గరిష్ఠ వయసు

Ping ఒక్కటే సరిపోదు. ఒక connection **చాలా రోజులు** బతికితే, అది database వైపు memory పేరుస్తుంది (prepared statements, temp tables), మరియు మీరు database ని replace చేసినా పాత దానికే అంటుకుని ఉంటుంది.

కాబట్టి ఒక **గరిష్ఠ వయసు** పెట్టాలి – అది దాటితే, ఖాళీగా ఉన్నప్పుడు నిశ్శబ్దంగా మూసేసి కొత్తది తెరవాలి:

```
#takeUsable() {
  while (this.free.length) {
    const c = this.free.pop();
    if (this.sim.now - c.bornAt > this.maxLifetimeMs) {   // వయసు మీరింది
      this.stats.retired++;
      this.retire(c);
      this.free.push(this.make());
      continue;                                           // తర్వాతిది చూడు
    }
    return c;
  }
  return null;
}
```

ఇది **ఖాళీగా ఉన్నప్పుడే** జరుగుతుంది కాబట్టి ఎవరికీ ఆలస్యం కాదు. మరియు ఇది pool ని **నెమ్మదిగా, నిరంతరం తాజా చేస్తుంది** – ఒకేసారి అన్నీ కాకుండా.

మూడో విరుపు పూర్తయింది. ఇప్పుడు అన్నీ కలిపి ఒక system చేద్దాం.

Part 5 – పూర్తి system

12. Step – pool పరిమాణం ఎంత ఉండాలి?

ఇది interview lo దాదాపు ఖచ్చితంగా వచ్చే ప్రశ్న, మరియు దీనికి **అందరూ ఇచ్చే సమాధానం తప్పు.**

సహజమైన ఆలోచన: "requests తిరస్కరించబడుతున్నాయా? pool ని పెద్దది చెయ్యి."

కొలుద్దాం. Demand సెకనుకు 1,000 queries. Database ఒకేసారి 12 queries వరకు బాగా భరిస్తుంది; అంతకు మించి ప్రతి అదనపు query అన్నిటినీ 3 ms నెమ్మది చేస్తుంది (lock contention, disk, CPU – నిజమైన databases ఇలాగే ప్రవర్తిస్తాయి).

```
demand 1,000 queries/సెకను · DB 12 కంటే ఎక్కువ ఒకేసారి భరించదు

  pool పరిమాణం | పూర్తయినవి | తిరస్కృతం | p50 | p99  | throughput
  -------------+---------+-----------+-----+------+-----------
       4      |     800 |      2200 | 1020 | 1020 |      200/s
       8      |    1600 |      1400 | 1020 | 1020 |      399/s
      12      |    2400 |       600 |  820 | 1020 |      598/s
      16      |    2012 |       988 | 1021 | 1032 |      499/s
      24      |    1737 |      1263 | 1056 | 1056 |      428/s
      40      |    1574 |      1426 | 1104 | 1104 |      384/s
      80      |    1507 |      1493 | 1224 | 1224 |      357/s
```

THROUGHPUT నిలువు వరుసని పైనుంచి కిందకి చదవండి

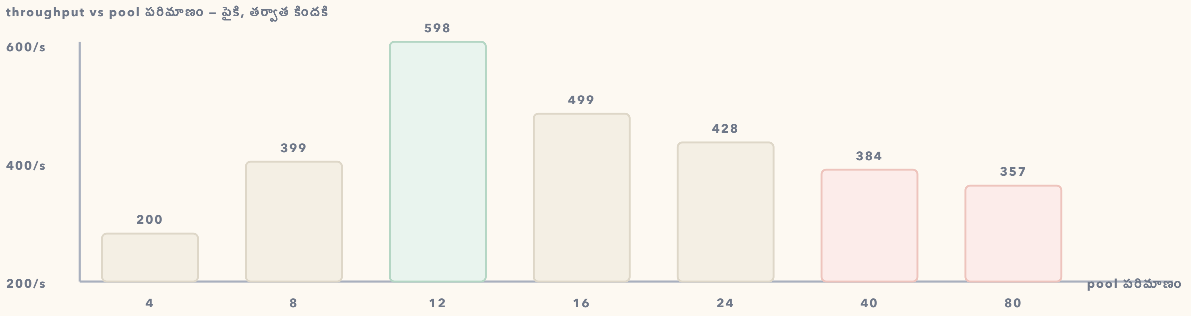
200 → 399 → **598** → 499 → 428 → 384 → **357**

అత్యుత్తమ స్థానం **pool = 12**, సరిగ్గా database యొక్క సామర్థ్యం.

మరియు **pool = 80 తో throughput 357/s** – pool = 12 కంటే **40% తక్కువ**. Pool ని 6.7 రెట్లు పెద్దది చేసి, మీరు system ని **నెమ్మది** చేశారు.

ఎందుకు? ఎందుకంటే **pool ఒక queue కాదు, అది ఒక అడ్డుకట్ట** (§1 lo చెప్పినది).

Pool చిన్నదిగా ఉంటే, అదనపు requests **బయట** ఎదురుచూస్తాయి – database ప్రశాంతంగా, వేగంగా పని చేస్తుంది. Pool పెద్దదిగా ఉంటే, అదే requests **database లోపల** ఎదురుచూస్తాయి – మరియు అక్కడ అవి ఒకదానికొకటి **నెమ్మది చేసుకుంటాయి**.



Pool చిన్నదైతే – అదనపు requests **బయట** ఎదురుచూస్తాయి, database ప్రశాంతం.

Pool పెద్దదైతే – అవే requests **database లోపల** ఎదురుచూస్తాయి, ఒకదాన్నొకటి నెమ్మది చేస్తూ.

ఎదురుచూపు పోలేదు. అది **మీరు చూడలేని చోటికి** మారింది – మరియు అక్కడ అది ఖరీదైనది.

INTERVIEW LO ఈ ప్రశ్నకి సమాధానం

"Pool పరిమాణాన్ని మీ app యొక్క traffic నుంచి లెక్కించకూడదు. దాన్ని database ఎంత ఏకకాలిక పని భరించగలదో దాని నుంచి లెక్కించాలి."

అంటే ఆ సంఖ్య **మీది కాదు, DBA ది**. మరియు అన్ని app instances కలిపి ఆ పరిమితిలో ఇమడాలి – 20 instances × pool 20 = database మీద 400 connections.

ఒక ఆచరణాత్మక సూత్రం: **connections ≈ (CPU cores × 2) + disk spindles**. 8-core database కి అది దాదాపు 20. చాలామంది 200 పెడతారు, మరియు వాళ్ళ system **నెమ్మదిగా** ఉంటుంది.

---

13. మొత్తం code · 3,000 యాదృచ్ఛిక ప్రయోగాలు · నాలుగు నియమాలు

ఇప్పుడు మూడు Parts lo నేర్చుకున్నవన్నీ ఒక చోట. ఇది పూర్తి code – ఏమీ దాచలేదు.

```
const NO_CONN = { reason: 'POOL_TIMEOUT' };

class Pool {
  #sim; #open; #live = []; #free = []; #waiters = [];
  #out = new Map(); #epoch = 0; #closed = false;
  #size; #borrowTimeoutMs; #idleValidateMs; #maxLifetimeMs; #leakAfterMs;
  #stats = { borrowed: 0, timedOut: 0, validated: 0,
             replacedStale: 0, retired: 0, leaksReclaimed: 0 };

  constructor(sim, {
    open, size = 10, borrowTimeoutMs = 2000,
    idleValidateMs = Infinity, maxLifetimeMs = Infinity, leakAfterMs = Infinity,
  }) {
    this.#sim = sim; this.#open = open; this.#size = size;
    this.#borrowTimeoutMs = borrowTimeoutMs;
    this.#idleValidateMs = idleValidateMs;
    this.#maxLifetimeMs = maxLifetimeMs;
    this.#leakAfterMs = leakAfterMs;
    for (let i = 0; i < size; i++) this.#free.push(this.#make());
  }
```

```
#make() {
  const c = this.#open(this.#sim.now);
  c.bornAt = this.#sim.now;
  c.idleSince = this.#sim.now;
  this.#live.push(c);
  return c;
}
#retire(c) {
  const i = this.#live.indexOf(c);
  if (i >= 0) this.#live.splice(i, 1);
  if (c.close) c.close();
}
```

§8 – ప్రధాన API. `release` ని మర్చిపోవడం సాధ్యం కాదు:

```
use(work, done = () => {}) {
  this.borrow((err, c) => {
    if (err) return done(err, null);
    let released = false;
    const finish = (e, v) => {
      if (released) return;                    // రెండోసారి పిలిస్తే పట్టించుకోవద్దు
      released = true;
      this.release(c, { broken: !!e });
      done(e, v);
    };
    try { work(c, finish); }
    catch (e) { finish(e, null); }             // విసిరినా connection వెనక్కి
  });
}
```

§5 – గడువుతో borrow. ఖాళీ లేకపోతే FIFO వరుసలో, కానీ ఎప్పటికీ కాదు:

```
borrow(cb) {
  if (this.#closed) return cb({ reason: 'POOL_CLOSED' }, null);
  this.#stats.borrowed++;
  const c = this.#takeUsable();
  if (c) return this.#check(c, cb);
  const w = { cb, dead: false };
  this.#waiters.push(w);                       // FIFO — మొదట అడిగినవాడికి మొదట
  if (this.#borrowTimeoutMs === Infinity) return;
  this.#sim.after(this.#borrowTimeoutMs, () => {
    if (w.dead) return;
    w.dead = true; this.#stats.timedOut++;
    this.#waiters.splice(this.#waiters.indexOf(w), 1);
    cb(NO_CONN, null);
  });
}
```

§11 – వయసు మీరినవి, మరియు ఖాళీగా కూర్చున్నవి:

```
#takeUsable() {
  while (this.#free.length) {
```

```
      const c = this.#free.pop();
      if (this.#sim.now - c.bornAt > this.#maxLifetimeMs) {   // వయసు మీరింది
        this.#stats.retired++;
        this.#retire(c);
        this.#free.push(this.#make());
        continue;
      }
      return c;
    }
    return null;
  }

  #check(c, cb) {                              // ఖాళీగా ఉంటేనే ping
    if (this.#sim.now - c.idleSince <= this.#idleValidateMs)
      return this.#hand(c, cb);
    this.#stats.validated++;
    if (c.alive) return this.#hand(c, cb);
    this.#stats.replacedStale++;
    this.#retire(c);
    this.#hand(this.#make(), cb);
  }
```

§8 – token తో reaper. ఇదే నేను మొదట తప్పుగా రాసినది:

```
  #hand(c, cb) {
    const token = ++this.#epoch;               // ఈ checkout కి మాత్రమే
    this.#out.set(c, token);
    if (this.#leakAfterMs !== Infinity)
      this.#sim.after(this.#leakAfterMs, () => {
        if (this.#out.get(c) !== token) return;    // పాత timer — వదిలేయ్
        this.#stats.leaksReclaimed++;
        this.#out.delete(c);
        this.#retire(c);                           // లీక్ అయినది నమ్మకూడదు
        this.#giveBack(this.#make());
      });
    cb(null, c);
  }

  release(c, { broken = false } = {}) {
    if (!this.#out.delete(c)) return;          // reaper ముందే తీసుకుంది
    if (broken) { this.#retire(c); this.#giveBack(this.#make()); return; }
    c.idleSince = this.#sim.now;
    this.#giveBack(c);
  }

  #giveBack(c) {
    while (this.#waiters.length) {
      const w = this.#waiters.shift();
      if (w.dead) continue;
      w.dead = true;
      return this.#check(c, w.cb);             // waiter కీ అదే పరీక్ష
    }
    this.#free.push(c);
  }
```

మూసివేత మరియు లెక్కలు:

```
  drain() {
    this.#closed = true;
    while (this.#waiters.length) {
      const w = this.#waiters.shift();
      if (!w.dead) { w.dead = true; w.cb({ reason: 'POOL_CLOSED' }, null); }
    }
    for (const c of [...this.#live]) this.#retire(c);
    this.#free = [];
  }

  stats() {
    return { ...this.#stats, size: this.#size, free: this.#free.length,
             inUse: this.#out.size, live: this.#live.length,
             waiting: this.#waiters.length };
  }
}
```

నాలుగు నియమాలు – మరియు వాటిని ఎలా పరీక్షించాలి

Unit tests రాయడం కంటే మంచి పద్ధతి ఒకటి ఉంది: **ఎప్పుడూ నిజం కావాల్సిన నియమాలను** రాసి, వాటిని **యాదృచ్ఛిక ప్రయోగాలతో** పరీక్షించడం.

#	నియమం	ఇది విరిగితే అర్థం
1	`live === size`	connections లీక్ అయ్యాయి, లేదా pool పరిమితి దాటి పెరుగుతోంది
2	`free + inUse === live`	ఒక connection లెక్కలో తప్పిపోయింది లేదా రెండుసార్లు లెక్కింది
3	ఒకే connection ఒకేసారి ఇద్దరికి ఉండకూడదు	queries ఒకదానితో ఒకటి కలిసిపోతున్నాయి
4	`free > 0` అయితే `waiting === 0`	ఖాళీ ఉండగా ఎవరో ఎదురుచూస్తున్నారు – అప్పగింత విరిగింది

Test ఒక్కో ప్రయోగంలో 400 requests పంపుతుంది, మరియు ప్రతిదీ యాదృచ్ఛికంగా ఇలా ప్రవర్తిస్తుంది:

```
pool.use((c, finish) => {
  if (seenBy.has(c.id)) bad.push(`connection ${c.id} ఇద్దరికి ఇచ్చారు`);
  if (c.closed)         bad.push(`మూసేసిన connection ${c.id} ఇచ్చారు`);
  seenBy.set(c.id, i);
  if (r < 0.06) { seenBy.delete(c.id); throw new Error('పని విసిరింది'); }
  sim.after(hold, () => {
    seenBy.delete(c.id);
    if (r < 0.12)      { /* లీక్: finish పిలవడం లేదు */ }
    else if (r < 0.18) { finish(new Error('connection చెడింది')); }
    else if (r < 0.24) { finish(null, 'ok'); finish(null, 'మళ్ళీ'); }
    else               { finish(null, 'ok'); }
  });
});
```

అంటే: 6% exception విసురుతాయి, 6% లీక్ అవుతాయి, 6% "చెడిపోయాయి" అని చెప్తాయి, 6% **రెండుసార్లు** `finish` పిలుస్తాయి. Pool పరిమాణం, గడువు, ping పరిమితి, గరిష్ఠ వయసు – అన్నీ ప్రతి ప్రయోగంలోనూ

యాదృచ్ఛికం. మరియు **25% connections పుట్టుకతోనే చచ్చినవి**, §10 lo చూసిన firewall కథని అనుకరిస్తూ.

```
3,000 యాదృచ్ఛిక ప్రయోగాలు · 6,74,115 పనులు పూర్తి · 4,76,407 తిరస్కృతం
  నియమ ఉల్లంఘనలు: 0
```

కానీ – ఈ test అసలు ఏదైనా పట్టుకోగలదా?

విఫలం కాలేని test ఏమీ నిరూపించదు. రెండు విధాలుగా తనిఖీ చేశాను.

ఒకటి – దారులన్నీ నిజంగా నడిచాయా?

```
ఏ దారులు నిజంగా నడిచాయి (3,000 ప్రయోగాల మొత్తం):
  borrowed         12,00,000
  timedOut          3,77,627
  validated            54,208
  replacedStale         8,616
  retired               2,874
  leaksReclaimed       49,478
```

ప్రతి కొమ్మా నడిచింది – 3.7 లక్షల గడువులు, 49,478 లీక్ మరమ్మత్తులు, 8,616 చచ్చిన connections replace అయ్యాయి.

రెండు – కావాలని విరగ్గొడితే పట్టుకుంటుందా? §8 lo సరిచేసిన ఆ ఒక్క పంక్తిని తీసేసి చూశాను:

```
==== token check తీసేస్తే fuzz పట్టుకుంటుందా? ====
3,000 యాదృచ్ఛిక ప్రయోగాలు · 9,55,576 పనులు పూర్తి · 1,74,839 తిరస్కృతం
  విఫలమైనవి: 3000
  ఉదా (seed 1): 812ms: live=8 కానీ size=6
```

3,000/3,000 ప్రయోగాలు విఫలం, మొదటి ప్రయోగంలోనే 0.8 సెకన్ల దగ్గర పట్టుకుంది.

> **మరియు ఇక్కడ ఒక ముఖ్యమైన వివరం ఉంది**
>
> విరిగిన version **ఎక్కువ పని చేసినట్టు** కనిపిస్తోంది – 9,55,576 vs 6,74,115. అది **42% ఎక్కువ**.
>
> ఎందుకంటే అది pool ని పరిమితి దాటి పెంచేస్తోంది. ఎక్కువ connections = ఎక్కువ throughput = **మంచి dashboard**.
>
> **ఈ bug ఒక మెరుగుదలలా కనిపిస్తుంది.** Performance metrics మాత్రమే చూస్తే, మీరు దీన్ని ఎప్పటికీ పట్టుకోలేరు. నియమాలే దీన్ని బయటపెట్టాయి.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ	ఏమి జోడించాం	ఏమి బాగుపడింది
§3	`free` జాబితా, `borrow` / `release`	pool అనేది ఉంది
§5	`timeoutMs`, `dead` గుర్తు, FIFO	1,990 ms → **గడువు మించదు**

ఎక్కడ	ఏమి జోడించాం	ఏమి బాగుపడింది
§8	`use()`, `released` గుర్తు	early return, `throw` — రెండింటిలోనూ లీక్ లేదు
§8	`token` తో reaper	52% వైఫల్యం → **0%**, మరియు లాగినవి = లీక్‌లు
§11	`idleSince` + షరతుతో ping	750 వైఫల్యాలు → **0**, 2,010 pings → **3**
§11	`maxLifetimeMs`	pool నెమ్మదిగా తనని తాను తాజా చేసుకుంటుంది
§13	`drain()`, `stats()`	మూసివేత, మరియు కనిపించడం

పూర్తి system – ఏ భాగం ఏ విరుపుని ఆపుతుంది

మూడూ ఒకే నిజాన్ని వేరే వైపుల నుంచి ఎదుర్కొంటాయి:

pool కి తన connections ఎక్కడ ఉన్నాయో తెలియదు.

కాబట్టి అది ఎప్పుడూ ఎదురుచూడకూడదు, ఎప్పుడూ నమ్మకూడదు, ఎప్పుడూ మర్చిపోకూడదు.

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెయ్యాలి
0–4	Pool ఎందుకు ఉంది – **రెండు కారణాలు**: వేగం, మరియు **database ని కాపాడటం**. రెండోది చెప్పండి, అది మిగతా అంతా నిర్ణయిస్తుంది
4–8	§2 ప్రశ్నలు. ముఖ్యంగా "**ఖాళీ అయితే ఏమి చెయ్యాలి?**" – దాన్ని interviewer తోనే నిర్ణయించండి
8–14	`free` జాబితా, `borrow` / `release` రాయండి. `undefined` సమస్యని **మీరే** చూపించండి
14–22	**మొదటి విరుపు.** "ఎదురుచూద్దాం" అని రాసి, అది ఆలస్యాన్ని అపరిమితం చేస్తుందని చూపించండి. గడువు జోడించండి
22–30	**రెండో విరుపు.** Early-return code ని board మీద రాసి లీక్ చూపించండి. `use()` ఇవ్వండి. తర్వాత "అయినా ఎవరో `borrow` నేరుగా వాడతారు" అని reaper జోడించండి
30–36	**మూడో విరుపు.** "రాత్రి 2 గంటలకి NAT gateway..." – ఖాళీగా ఉంటేనే ping అని చెప్పండి
36–42	**Pool పరిమాణం.** "పెద్దది చేస్తే ఘోరం" – ఇది interviewer కి ఆశ్చర్యం కలిగిస్తుంది
42–45	నియమాలు. "నేను unit tests కంటే ఈ నాలుగు నియమాలని యాదృచ్ఛిక ప్రయోగాలతో పరీక్షిస్తాను"

ఏమి తప్పక చెప్పాలి

1. **Pool ఒక limiter, cache కాదు.** ఇది ఒక్క వాక్యం, మరియు ఇది మీ మొత్తం సమాధానాన్ని వేరే స్థాయిలో ఉంచుతుంది.
2. **అపరిమిత ఎదురుచూపు ఒక bug.** గడువు లేని pool "అన్నీ విజయవంతం" అని చెప్తుంది, కానీ latency పెరుగుతూనే ఉంటుంది.
3. **`borrow` / `release` ని బహిరంగంగా ఇవ్వకూడదు.** `use()` మాత్రమే. "సరిగ్గా వాడటం తప్ప వేరే దారి లేని API."
4. **Connections ఖాళీగా కూర్చోవడం వల్ల చస్తాయి, వాడటం వల్ల కాదు.** అందుకే ప్రతిసారీ కాదు, ఖాళీగా ఉంటేనే ping.
5. **Pool పరిమాణం database యొక్క సంఖ్య, మీ app ది కాదు.**

ఏమి వదిలేయాలి

- Thread-safety యొక్క వివరాలు – ఒక వాక్యంలో చెప్పి ముందుకి వెళ్ళండి
- కనిష్ఠ/గరిష్ఠ పరిమాణం తో pool పెరగడం/తగ్గడం – అడిగితే మాత్రమే
- Prepared statement cache – వేరే problem
- పరిమాణాన్ని తనంతట తానే సర్దుకునే pool – చాలా systems lo ఇది స్థిరంగానే ఉంటుంది, మరియు అదే మంచిది

15. నోటితో చెప్పాల్సిన English script

> "A connection pool looks like a cache, but it's really a **limiter**. Opening a connection costs a TCP handshake plus TLS plus auth — about thirty milliseconds against a two-millisecond query. But the more important job is that the database has a hard connection ceiling, and the pool is what keeps my app from crossing it."

> "I'll start with a free list and `borrow` / `release`. When it's empty, `pop()` returns `undefined`, and the app crashes with a `TypeError` about a missing `query` method — which hides the real cause. So the first fix is to **name the failure at the point it happens**."

> "The natural next step is to queue and wait. That's a trap. I measured it: at twenty percent over capacity, five hundred requests wait up to 490 milliseconds and two thousand requests wait up to **1,990**. It scales with arrivals, so it never settles. And every request still 'succeeds' — the error dashboard stays green while users wait two seconds. **Waiting isn't a fix, it's a way of hiding the problem.** So I put a timeout on the wait, which caps latency at exactly the timeout regardless of load."

> "The second failure is leaks. Any early return or thrown exception between borrow and release loses a connection **permanently** — the object is still alive in a closure, so GC won't help. I measured it: with a pool of ten and four thousand requests, **nine leaks rejected fifty-two percent of traffic** and killed the pool in 4.4 seconds. A 0.22% error rate became a 52% failure rate."

> "Two fixes. First, don't expose `borrow` and `release` at all — expose `use(work)` that releases in a `finally`. Second, reclaim connections held too long. I got that one wrong initially: my reaper scheduled a new timer on every hand-off but never cancelled the old one, so a stale timer would yank a connection that someone else was actively using. **It still showed 100% success** — I only caught it because I counted reclaims and they were 391 against 20 leaks. The fix is a per-checkout token the timer checks before acting."

> "Third failure is stale connections. During a quiet night, idle connections get closed by NAT or a firewall, and TCP doesn't tell you – the socket still looks connected. I measured a sixty-second quiet period: the firewall killed connections at thirty-five seconds, but the **first failure landed at sixty-five seconds**, the exact moment traffic returned. Thirty seconds broken with zero errors reported. That's the Monday-morning outage."

> "Validating on every borrow fixes it but costs a round trip per query. The insight is that connections die from **sitting idle**, not from being used – so I only validate when idle time exceeds a threshold. Same zero failures, **three pings instead of 2,010**."

> "On sizing: the instinct is to grow the pool when requests get rejected. I measured the opposite. Against a database that degrades past twelve concurrent queries, throughput peaks at pool size twelve – and **a pool of eighty delivers forty percent less throughput**. A bigger pool doesn't remove the waiting, it just moves it inside the database where it's more expensive. So the pool size comes from what the database can take, not from my traffic."

> "For testing I'd write four invariants – `live === size`, `free + inUse === live`, no connection handed to two callers, and never anyone waiting while something's free – then fuzz them. Three thousand randomized runs with injected leaks, exceptions, and double-releases: zero violations. And when I removed the token check on purpose, it failed all three thousand – while reporting **42% more throughput**, because the bug inflates the pool. That's why I test invariants and not just metrics."

---

16. Follow-ups – circuit breaker, బహుళ databases, async/await

"Database పూర్తిగా పడిపోతే?"

అప్పుడు గడువు కూడా సరిపోదు. ప్రతి request 2 సెకన్లు ఎదురుచూసి విఫలమవుతుంది – అంటే users నెమ్మదిగా విఫలమవుతారు, ఇది వెంటనే విఫలమవడం కంటే ఘోరం. కావాల్సింది **circuit breaker**: వరుసగా N వైఫల్యాలు వస్తే pool ని "తెరిచిన" స్థితిలో పెట్టి, **ఎదురుచూపు లేకుండానే** తిరస్కరించండి. కొంత సమయం తర్వాత ఒక్క request ని పరీక్షగా పంపండి.

"Read replica కూడా ఉంటే?"

ఒక pool కాదు, **ఒక్కో లక్ష్యానికి ఒకటి**. Primary కి ఒకటి, ప్రతి replica కి ఒకటి. కారణం §12 – పరిమాణం ఆ *database* యొక్క సామర్థ్యం నుంచి వస్తుంది, మరియు అవి వేర్వేరుగా ఉంటాయి. వాటి మీద ఒక routing పొర పెట్టండి: `SELECT` → replica, మిగతావి → primary.

"Pool పరిమాణాన్ని తనంతట తానే మార్చుకోవచ్చా?"

చేయగలం, కానీ **చాలా జాగ్రత్త**. Load పెరిగినప్పుడు pool పెరిగితే, §12 lo చూసినట్టు అది database ని **ఇంకా నెమ్మది** చేస్తుంది – అది latency ని పెంచుతుంది, అది "ఇంకా pool కావాలి" అనిపిస్తుంది. **ఒక దుష్ట చక్రం.** కనిష్ఠాన్ని మాత్రమే మార్చడం సురక్షితం (ఖాళీగా ఉన్నవాటిని రాత్రిపూట తగ్గించడం), గరిష్ఠాన్ని స్థిరంగా ఉంచడం.

"ఇది `async` / `await` తో ఎలా ఉంటుంది?"

ఈ doc lo callbacks వాడాను ఎందుకంటే సిమ్యులేషన్ సమయాన్ని నేనే నియంత్రిస్తున్నాను. నిజమైన code lo ఆకారం ఇది, మరియు **logic ఒక్కటే**:

```
async use(work) {
  const c = await this.borrow();          // గడువు దాటితే throw
  try { return await work(c); }
  finally { this.release(c); }            // ← ఇక్కడే మొత్తం §8
}
```

ఆ `finally` ఒక్క పంక్తే §8 మొత్తం. కానీ **reaper ఇప్పటికీ కావాలి** – ఎందుకంటే `work` ఎప్పటికీ పూర్తవకపోతే (ఒక hung socket) `finally` ఎప్పటికీ నడవదు.

"Connection మీద transaction ఉంటే?"

`use()` బ్లాక్ ఒక transaction యొక్క సరిహద్దు కావాలి. `release` కి ముందు `ROLLBACK` పంపడం ముఖ్యం – పని విఫలమైతే ఆ connection మీద ఒక తెరిచిన transaction ఉండిపోతుంది, మరియు అది తర్వాతి వ్యక్తికి వెళ్తుంది. `broken: true` దారిలో connection ని పూర్తిగా మూసేయడం ఇంకా సురక్షితం.

---

17. ఏమి నేర్చుకున్నాం

1. Pool ఒక cache కాదు, ఒక limiter. వేగం దాని దుష్ప్రభావం మాత్రమే. దాని అసలు పని database ని మీ స్వంత traffic నుంచి కాపాడటం. ఈ ఒక్క వాక్యం §5, §12 రెండింటినీ వివరిస్తుంది.

2. ఎదురుచూపు ఒక పరిష్కారం కాదు. గడువు లేని pool 100% విజయం చూపిస్తూ latency ని అపరిమితంగా పెంచుతుంది. **"అంతా విజయవంతం" అనే metric ఒక hazard కావచ్చు.**

3. సరిగ్గా వాడటం మీద ఆధారపడే API ఇవ్వకండి. `borrow` / `release` ని బహిరంగంగా ఇస్తే ఎవరో ఒకరు మర్చిపోతారు – మరియు 0.22% లోపం 52% వైఫల్యం అవుతుంది. `use()` మాత్రమే ఇవ్వండి.

4. వనరులు వాడటం వల్ల చావవు, ఖాళీగా కూర్చోవడం వల్ల చస్తాయి. ఈ ఒక్క అంతర్దృష్టి 2,010 pings ని 3 కి తగ్గించింది – అదే రక్షణతో.

5. విరిగిన క్షణానికీ, తెలిసిన క్షణానికీ మధ్య అగాధం ఉంటుంది. మరియు ఆ అగాధం చివర ఎప్పుడూ **traffic తిరిగి వచ్చే క్షణం** ఉంటుంది. అందుకే systems సోమవారం ఉదయం విరుగుతాయి.

6. ఎక్కువ వనరులు ఎప్పుడూ మంచివి కావు. Pool ని 12 నుంచి 80 కి పెంచితే throughput **40% తగ్గింది**. ఎదురుచూపు పోలేదు – అది మీరు చూడలేని చోటికి మారింది.

7. సరైన ఫలితం సరైన కారణం వల్లేనా అని చూడండి. నా మొదటి reaper 100% విజయం చూపించింది, కానీ నడుస్తున్న queries నుంచి connections లాగేస్తోంది. అది బయటపడింది ఎందుకంటే **ఆ సంఖ్య ఎంత ఉండాలో నాకు ముందే తెలుసు** – లీక్‌ల సంఖ్యకి సమానంగా. ప్రతి మరమ్మత్తుకీ ఒక counter పెట్టండి, మరియు దాని ఆశించిన విలువను ముందే రాయండి.

8. Metrics కాదు, నియమాలు పరీక్షించండి. Token పరీక్ష తీసేసిన version **41% ఎక్కువ throughput** చూపించింది. Performance dashboard దాన్ని ఒక మెరుగుదలగా చూపేది. నాలుగు నియమాలు దాన్ని 0.8 సెకన్లలో పట్టుకున్నాయి.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Pool కి తన connections ఎక్కడ ఉన్నాయో ఎప్పటికీ తెలియదు.

అది వాటిని ఇచ్చిన క్షణం నుంచి అవి తిరిగి వచ్చే క్షణం వరకు, అవి బతికున్నాయో లేదో, తిరిగి వస్తాయో రావో pool కి తెలియదు.

కాబట్టి pool యొక్క ప్రతి design నిర్ణయం ఒకే ప్రశ్నకి సమాధానం: **"నాకు తెలియని దాని గురించి నేను ఏమి ఊహించకూడదు?"**

ఎప్పటికీ ఎదురుచూడకు (§5). ఎప్పటికీ తిరిగి వస్తుందని నమ్మకు (§8). ఎప్పటికీ బతికే ఉందని నమ్మకు (§11).